



火星编辑器小白指南

Markdown 一键变公众号排版：开源工具 Mars Editor 从原理到上手

官网 mars-editor.cc | 网页版 mars-editor.pages.dev | GitHub: [whyubel1eve/Mars-Editor](https://github.com/whyubel1eve/Mars-Editor) (网页版) · Mars-Editor-APP (桌面版) | 2026-10-02

一句话认识它

火星编辑器 = 一个 Markdown 编辑器 + 一个“微信保镖”转换器。 你用 Markdown 写文章，它把全文转成**每一个标签都自带内联样式的富文本**，复制后粘贴进微信公众号后台，样式一字不差。开源免费（MIT），有网页版（React）和桌面版（Tauri + Rust，支持 macOS / Windows / Linux），桌面版还内置本地文件防丢失和 Claude Code / Codex CLI 接入。

疑问清单（小白视角，从浅到深）

Q1

公众号排版到底难在哪？大家以前怎么干的？

微信公众号后台编辑器是个“残缺版 Word”：不支持 Markdown、粘贴外部富文本会掉样式、代码块没有高亮、数学公式基本没戏。写技术文章的人每发一篇都要手动调格式半小时。

老办法	痛点
后台手动排版	每篇 20-60 分钟，纯体力活
mdnice / 墨滴等在线工具	要联网、草稿存别人服务器，怕跑路丢稿
壹伴等浏览器插件	功能绑在插件生态里，不开放
火星编辑器	开源 + 草稿是本地 .md 文件 + 一键转换，三个痛点各补一刀

Q2

为什么粘贴进微信会“排版全丢”？微信编辑器对 HTML 做了什么？

微信后台用的是 UEditor 内核，粘贴时会做一次“消毒”（sanitize）：按白名单过滤标签和属性。**class 属性和 <style> 标签会被整段删掉，只留下每个标签自带的 style 内联样式。**所以普通网页那套“CSS 类 + 样式表”的排版在微信里全军覆没——样式表没了，类名也没了，正文只剩裸文字。

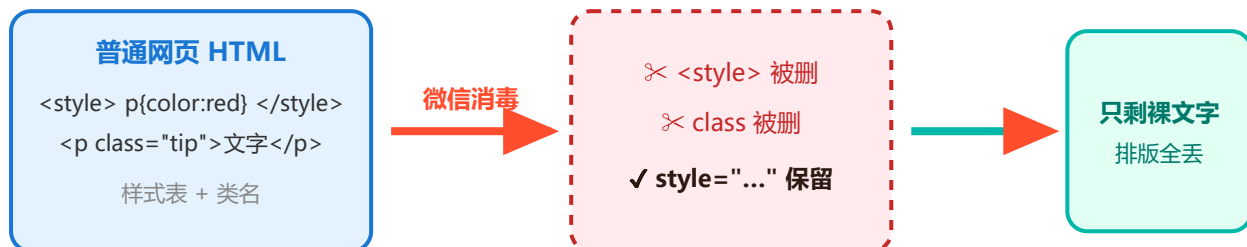


图 2-1：微信编辑器的“消毒”流水线——只有内联 style 能活着通过

Q3

“内联样式”是什么？为什么它能一字不差？

CSS 样式有三种挂法：**外链样式表**（.css 文件）、**页内 <style>**、**内联 style 属性**（直接写在标签上，如 `<p style="color:red">`）。微信只认第三种。火星编辑器把整篇文章**每个标题、每段、每个列表项都“焊”上一份完整的 style**——样式跟着标签走，不需要任何外部资源，粘贴到哪都原样。代价是 HTML 体积膨胀（本指南实验里一段样例就有 45 处 style），但公众号场景体积不是问题。

```
<!-- 普通写法（微信会丢） -->
<style>
  h2 { color:#ff4d2e }
</style>
<h2>标题</h2>

<!-- 内联写法（微信保留） -->
<h2 style="color:#ff4d2e;
font-size:24px">标题</h2>
```

类比：外链样式像“装修手册放在物业办公室”——进小区（微信）时手册被没收，房子立刻变毛坯；内联样式像**把装修直接烧进砖里**，房子搬哪都带装修。讲完类比落回术语：**style 内联属性是随标签自包含的样式声明，不依赖任何外部选择器。**

Q4 Markdown 是怎么一步步变成"内联样式 HTML"的?

核心是 `markdown-it` (JavaScript 生态最常用的 Markdown 解析器)。它先把 Markdown 切成 **Token 流** (一串"开门/关门"标记, 如 `paragraph_open / inline / paragraph_close`) , 默认规则输出普通 HTML。火星编辑器的做法是**把全部几十条渲染规则整包覆盖 (override)** , **每条规则都输出带主题配色的内联 style 版本** , 连脚注规则也覆盖了。主题 (9 套网页版 / 12 套桌面版) 只是配色数据, 渲染时通过 `env` 注入逐规则读取。

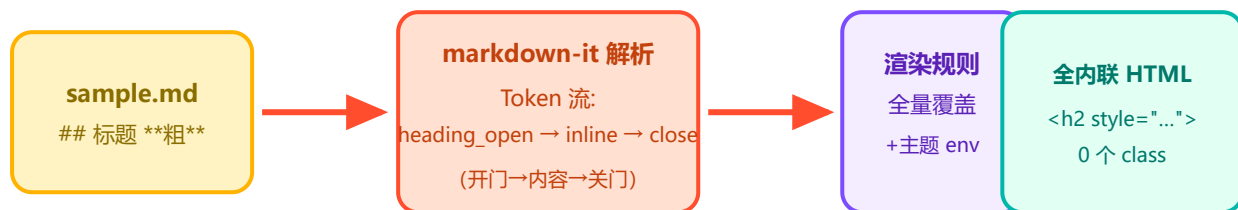


图 4-1: Markdown → Token 流 → 覆盖渲染规则 → 全内联样式 HTML

Q5 "复制富文本"是怎么实现的?

浏览器的剪贴板一次可以写多种格式。火星编辑器用 `ClipboardItem` **同时写入 `text/html` (那份内联样式 HTML) 和 `text/plain` (纯文本兜底)**。粘贴程序自己挑: 微信后台挑 `text/html`, 记事本挑 `text/plain`。老浏览器则回退到"隐藏 `contenteditable` 容器 + `execCommand('copy')`"。复制前还有一步细节: **剥掉 `data-line` 等预览专用标记** , 防止公众号把这些多余属性原样存下来。纯文本转换时给块级标签补换行, 否则整篇会挤成一行 (`DOMParser` 不参与排版, `innerText` 会退化成 `textContent`) 。

Q6 图片不放图床, 粘到微信里怎么不会挂?

常规做法是图片传到图床、正文里放外链——但微信有防盗链, 外链图经常裂。火星编辑器反着来: 网页版把图片存进浏览器 `IndexedDB` (浏览器自带的大容量键值数据库) , 渲染时转成 **`data URI` (把图片二进制 `base64` 编码后直接塞进 `src="data:image/png;base64,..."`)** , 图片本体跟着 HTML 走。实测粘贴进公众号后台后原样保留, 完全不需要图床。桌面版则是图片落工作区 `images/` 文件夹, 复制时自动内嵌、外链图自动转文末脚注绕防盗链。

⚠ 代价要懂: `base64` 会让体积膨胀约 33%, 图太多文章会重。这是"自包含换体积"的典型取舍。

Q7 网页版和桌面版差在哪？为什么桌面版要用 Tauri + Rust？

网页版受浏览器沙箱限制：摸不到本地文件系统、装不了文件监听、调不了本机命令行。**Tauri 2 是"Rust 外壳 + 系统 WebView + 前端页面"的桌面应用框架**——界面还是那套 React，但 Rust 后端给了它原生能力。对比 Electron：Electron 捆一个完整 Chromium（每个应用 100+MB），Tauri 用系统自带 WebView，安装包小一个数量级。

能力	网页版	桌面版 (Tauri)
Markdown → 内联样式	✓	✓ (同一套前端代码)
草稿存储	IndexedDB (浏览器里)	本地 .md 文件 (真文件系统)
主题	9 套	12 套
推公众号草稿箱	✗	✓ (AppID/AppSecret + IP 白名单)
接 Claude Code / Codex	✗	✓
网页转 Markdown 导入	✗	✓ (公众号走本地抓取，其他走 Jina Reader)

Q8 "本地文件防丢失"是怎么做到的？

桌面版的设计原则是**"草稿就是本地 .md 文件，应用只是文件的视图"**：文章存在你指定的文件夹里，编辑器改动直接写盘；反过来，任何程序（包括 AI）改了这些 .md 文件，编辑器通过**文件监听 (vault watch，类似前端 watchEffect 的"盯着文件系统"版)** 发现变化并重载界面。应用的数据库只存界面状态，**不碰 CLI 自己的会话文件 (~/.claude/projects/ 等内部格式，随时会变)**。这样即使火星编辑器明天删库跑路，你的稿子还是一堆普通 .md，任何编辑器都能打开。

Q9

它是怎么接入 Claude Code / Codex CLI 的？

桌面版顶栏的 Agent 面板没有接任何模型 API——它**直接把本机已装的 CLI 当子进程启动**（Rust 的 `std::process::Command`），用的是你自己的账号和配置，不用填 key：

```
claude -p "帮我把这篇改成口语化" --output-format stream-json --verbose
codex exec --json "帮我把这篇改成口语化"
```



图 9-1：Agent 接入 = spawn 本机 CLI + JSONL 流式转发 + 文件监听回读

两个 CLI 的输出事件格式完全不同，**Rust 侧不解析不合并，原样逐行转发，翻译成统一时间线是前端的活**——“谁需要理解，谁来做”。这与知识库 [[子智能体咨询]] 是同一模式：包装本机 CLI 而非自接 API。注意：Agent 功能要 CLI 版可执行文件，桌面 GUI 版不往 PATH 放可执行文件，检测不到时需另装 CLI。

Q10

代码里有哪些值得学的工程细节？

细节	做法	为什么妙
highlight.js 懒加载	动态 import，就绪前代码块按原文转义输出	数百 KB 的语言包不进首屏，冷启动快；加载完成后触发一次重渲染补高亮
零依赖 zip	用浏览器原生 <code>CompressionStream</code> 自己实现 zip 读写	备份/还原不引第三方库——知识库 [[零依赖编程]] 的实战版
导出长图	把正文 DOM 塞进 SVG <code><foreignObject></code> 序列化后转 750px 宽 PNG	不依赖 html2canvas，纯浏览器原生子方案
滚动同步	渲染时记录每个块对应的源码行号（data-line / map）	编辑区和预览区按行号对齐滚动
容错哲学	Agent 模块注释：“最坏也只是面板停了，稿子照样改”	关键路径（文件）与展示路径（面板）分离，坏一层不坏全局

Q11 "一键推送公众号草稿箱"是怎么回事？

微信公众号开放平台提供草稿箱 API。桌面版里填上公众号的 **AppID / AppSecret**（公众平台后台申请），并把本机 IP 加进公众号后台的 **IP 白名单**，即可把排版好的文章直接推成公众号草稿，省掉"复制→粘贴→检查"一步。小白注意：AppSecret 等同账号密钥，只存在自己机器，别截图外传。

Q12 它和我知道的工具是什么关系？

和 **mdnice/墨滴**：同类（Markdown→公众号），但火星编辑器开源可自部署 + 桌面版本地文件；和 **Typora/Obsidian**：它们是通用 Markdown 编辑器，火星编辑器只专注"写到公众号"这一条管道；和 **壹伴**：壹伴是插件生态，火星编辑器是独立应用 + 开源。与知识库已学内容的关联：它的 Agent 接入同 [[子智能体咨询]]；它覆盖 markdown-it 渲染规则的做法是 [[构建流水线与Pass]] 思想（解析 Pass → 渲染 Pass）在前端工具里的体现；懒加载 highlight.js 体现的是"按需加载"。

Q13 5 分钟上手路径是什么？

零安装：打开 **mars-editor.pages.dev**（网页版）→ 左边写 Markdown → 右边看排版 → 点复制 → 粘进公众号后台。进阶：官网 **mars-editor.cc** 下载桌面版（Windows 用 NSIS 安装包），草稿落本地文件夹，想要 AI 帮写再装 Claude Code 或 Codex CLI（PATH 里能找到即可被检测到）。

这个项目给我什么（能力清单）

#	能力	你能干什么
1	Markdown → 公众号内联样式富文本	技术笔记发公众号不再手动排版，粘贴即成
2	9-12 套主题 + 三档排版密度	一句话切整套视觉风格（实测三主题输出色值完全不同）
3	本地 .md 草稿 + 备份 zip 导入导出	稿子归自己，换工具零锁定
4	AI Agent 面板（桌面版）	Claude Code/Codex 直接在稿子文件夹里帮你改稿，编辑器实时刷新
5	推公众号草稿箱 / 导长图 PNG / 网页转 MD	发布管道一条龙：抓网页→AI 改写→排版→推草稿
6	一套可学习的干净前端工程	markdown-it 规则覆盖、剪贴板双格式、懒加载、Tauri 架构都是可迁移的套路

动手实验（全部真实运行，环境：Windows 11 + Node v24.21.0）

实验 1 跑起网页版

操作: `git clone https://github.com/whyubelieve/Mars-Editor repo && cd repo && npm install && npx vite --port 5199`

目的: 验证项目能本地跑起来。实现: Vite 开发服务器。

✅ 预期结果: 终端输出 `VITE v7.3.6 ready in 970 ms → Local: http://localhost:5199/` ; 浏览器打开即编辑器 (实测 `curl` 返回 HTTP 200, 页面 `lang="zh-CN"`) 。

坑: `npm install` 对 `workerd` 的 `postinstall` 脚本可能被安全策略拦 (exit 2) , 不影响 `dev/build`——`workerd` 只有 Cloudflare 部署才用。

实验 2 在 Node 里直接驱动它的渲染管线 (主实验)

操作: 项目没把渲染器暴露成 CLI, 就用它自带的 `esbuild` 把 `src/markdown.ts` 打包成 Node 可跑模块 (材料在 `exercise\`) :

```
../repo/node_modules/.bin/esbuild render_entry.ts --bundle --platform=node --format=esm --outfile=render_node render.mjs sample.md classicTheme
```

目的: 亲手验证核心卖点——输出 HTML 里没有 `class`、全部样式内联。实现: `render_entry.ts` 调用项目自己的 `renderArticle()` 渲染 13 行样例文章 (含标题/高亮/待办/提示条/代码块/脚注) , 并自动断言四项微信兼容性检查。

✅ 预期结果 (实测输出) :

PASS 正文不含 `class` 属性 (微信会丢弃)

PASS 正文不含 `<style>` 标签 (微信会丢弃)

PASS 正文不含 `<script>` 标签

PASS 块级标签全部带内联 `style` | 内联 `style` 总数: 45, 退出码 0

输出样例: `<h1 style="font-family:Georgia,...;font-size:28px;...">火星编辑器转换测试</h1>` ——注意 `data-line` 是预览标记, 正式复制前会被 `clipboard.ts` 剥掉。

实验 3 换主题看差异

操作: `node render.mjs sample.md sakuraTheme` 和 `node render.mjs sample.md darkTheme`

目的: 验证"主题=配色数据"不是宣传语。实现: 同一份 Markdown, 不同 Theme 对象进 `env`。

✅ 预期结果 (实测) : `classic` 正文色 `#2b2b2b` ; `sakura` 背景 `#fffafc`、高亮条 `#ffe0ec` (粉) ; `dark` 文字 `#c9d4c3`、代码 `#e6ffe9` 。三次运行退出码均为 0。

知识点地图 (入库概念)

本次新入库：[[内联样式与富文本转换]]（微信消毒机制 + markdown-it 规则覆盖 + 剪贴板双格式） | [[Tauri桌面应用框架]]（Rust 外壳 + 系统 WebView，对照 Electron） | [[本地优先与文件监听]]（草稿即本地文件 + watch 重载 + 不碰工具内部格式）。

关联已有：[[子智能体咨询]]（同为包装本机 Claude Code CLI）、[[零依赖编程]]（CompressionStream 自实现 zip）、[[构建流水线与Pass]]（解析/渲染两段式）。

火星编辑器小白指南 · 拾遗任务 1787578235981 · learn-project 流程产出 · 2026-10-02 · 实验数据均来自本机真实运行